



KubeCon



CloudNativeCon

Europe 2026

Three Shades of Isolation: A multi-tenancy Fortress

Braulio Dumba & Paolo Dettori, IBM Research



Multi-Tenancy in Kubernetes

- **Definition**

- Sharing cluster resources among multiple tenants (users, teams, applications)
- Growing demand as Kubernetes workloads move to production

- **Key Requirements**

- *Data-Plane Isolation*: tenants do not share compute nodes
- *Access Boundaries*: logical segregation via network policies, namespaces
- *Control-Plane Isolation*: each tenant gets their own control plane
- *Vanilla K8s Experience*: tenants manage own CRDs, ClusterRoles, StorageClasses

Motivation: Why Isolate Workloads?

- **Security**

- Prevent cross-tenant access to pods, deployments, and secrets
- Enforce access boundaries with logical and physical segregation

- **Performance**

- Eliminate the “noisy neighbor” problem
- One tenant’s high scheduling throughput should not impact another

- **Cluster-Scoped Resource Conflicts**

- Cluster-level objects (CRDs, ClusterRoles, StorageClasses) cannot be shared across deployments
- AI platforms (e.g., *kagenti*, *llm-d*, *vLLM*) need strong tenant isolation for shared GPU infrastructure

Multi-Tenancy Models & Their Limitations

- **Cluster-as-a-Service**
 - Each tenant gets a dedicated cluster with full isolation
 - **Limitation:** cluster sprawl, high operational cost, poor utilization
- **Namespace-as-a-Service (e.g., Capsule)**
 - Tenants share a cluster; confined to dedicated namespaces
 - **Limitation:** no cluster-scoped resources, noisy neighbor, complex RBAC
- **Control-Plane-as-a-Service (e.g., vcluster)**
 - Each tenant gets a virtual cluster with dedicated control plane
 - **Limitation:** still shares data-plane (worker nodes) across tenants

New Approach: Three Shades of Isolation

- **Design Goals**

- Easy management (i.e., simple management & enforcement strategies)
- Cost effective (i.e., minimal resource overhead)
- Cloud-native open-source technologies

- **Isolation Layers**

- Data-plane
- Control-plane
- Network: decoupled control & data planes

Building Blocks: Open-Source Technologies

KubeFlex (KubeStellar)

- Provides dedicated control planes per tenant
- Scalable multi-cluster config management

KubeVirt

- Runs VMs as Kubernetes pods for data-plane isolation
- Cost-effective alternative to physical node isolation

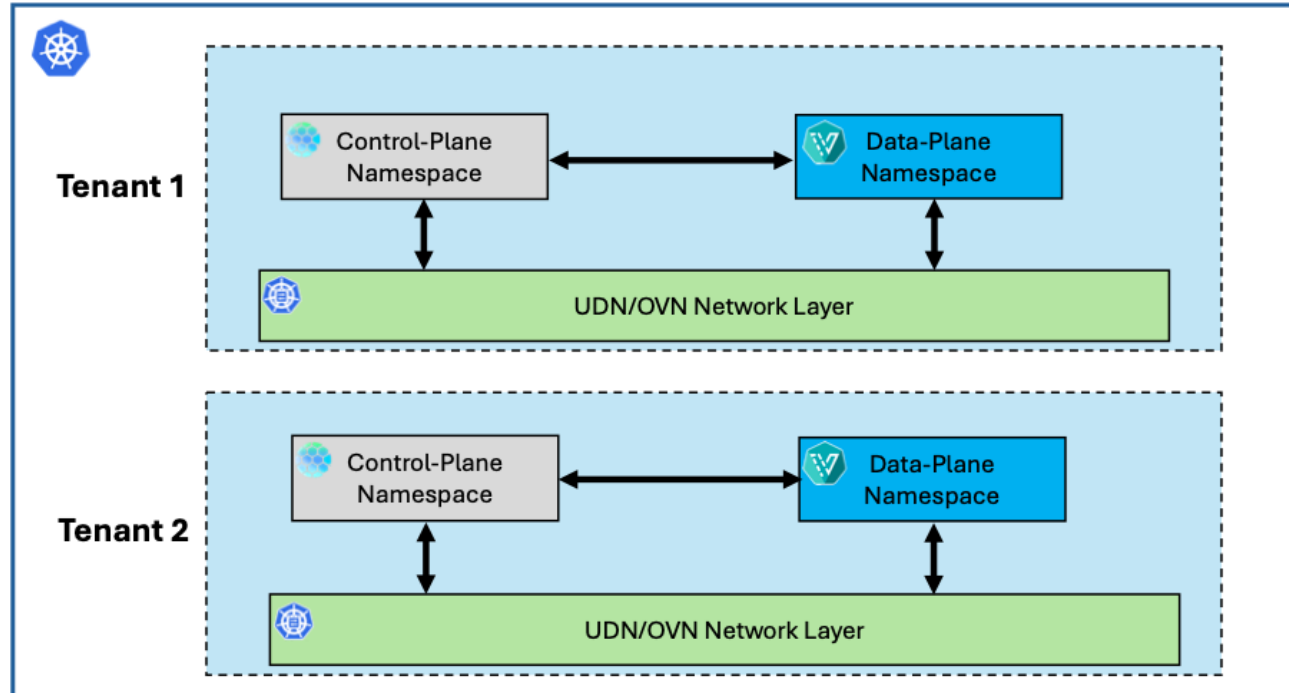
K3s

- Lightweight certified K8s in a single binary
- K3s agent connects worker nodes to control plane efficiently

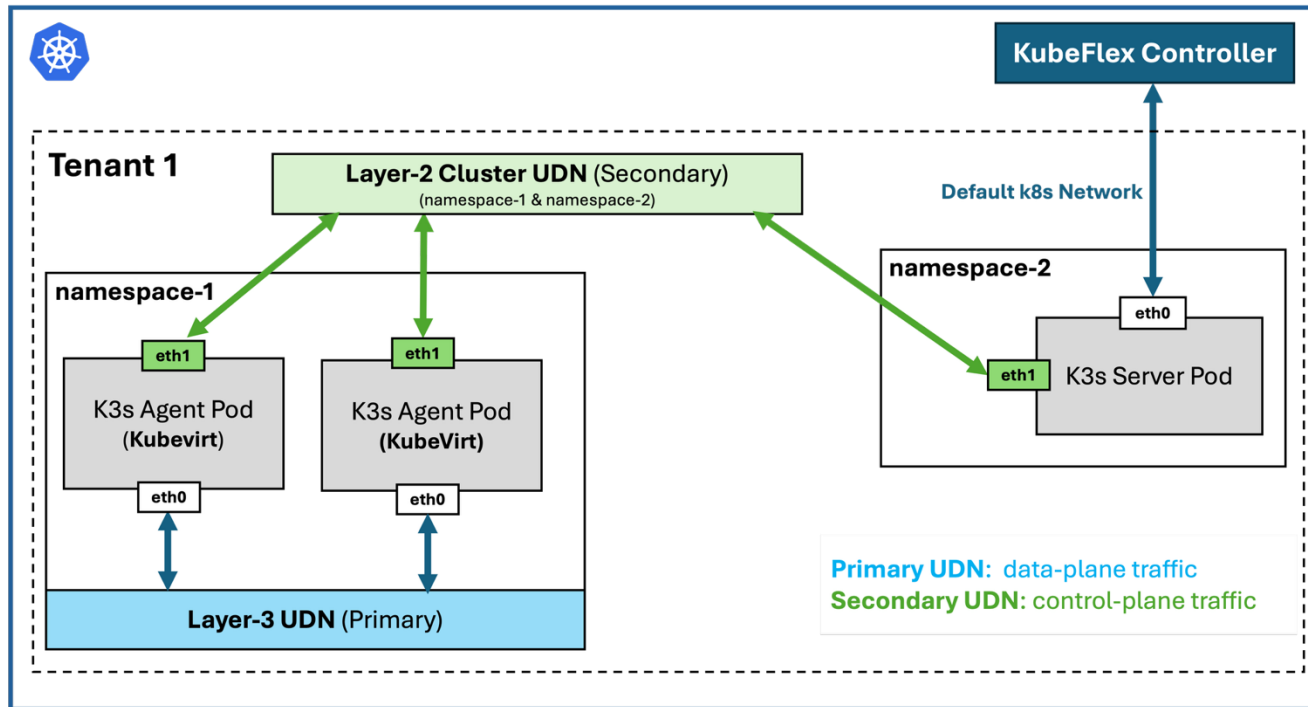
UDN / OVN-Kubernetes

- User Defined Networks for network-level tenant isolation
- Layer 2/3 segmentation between tenant traffic

New Approach: Overview



New Approach: Framework Architecture



New Approach: Demo – Infrastructure



KubeCon



CloudNativeCon

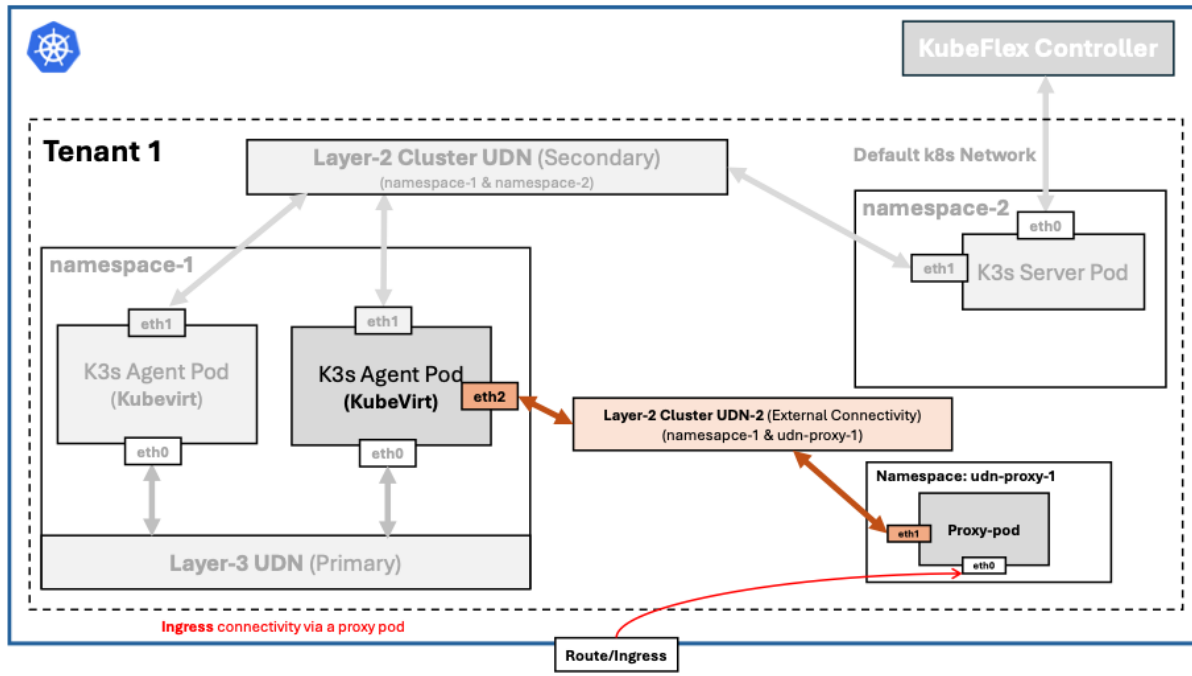
Europe 2026

Setup

- Hosted control plane K8s cluster
- 5 Worker nodes (72 CPUs, 755 GB RAM per node)

Workloads

- *Tenant-1*: llm-d-sim[1]
- *Tenant-2*: vllm production-stack[2]



Why proxy? Ingress connects to the default network. A proxy pod bridges the default and UDN tenant networks.



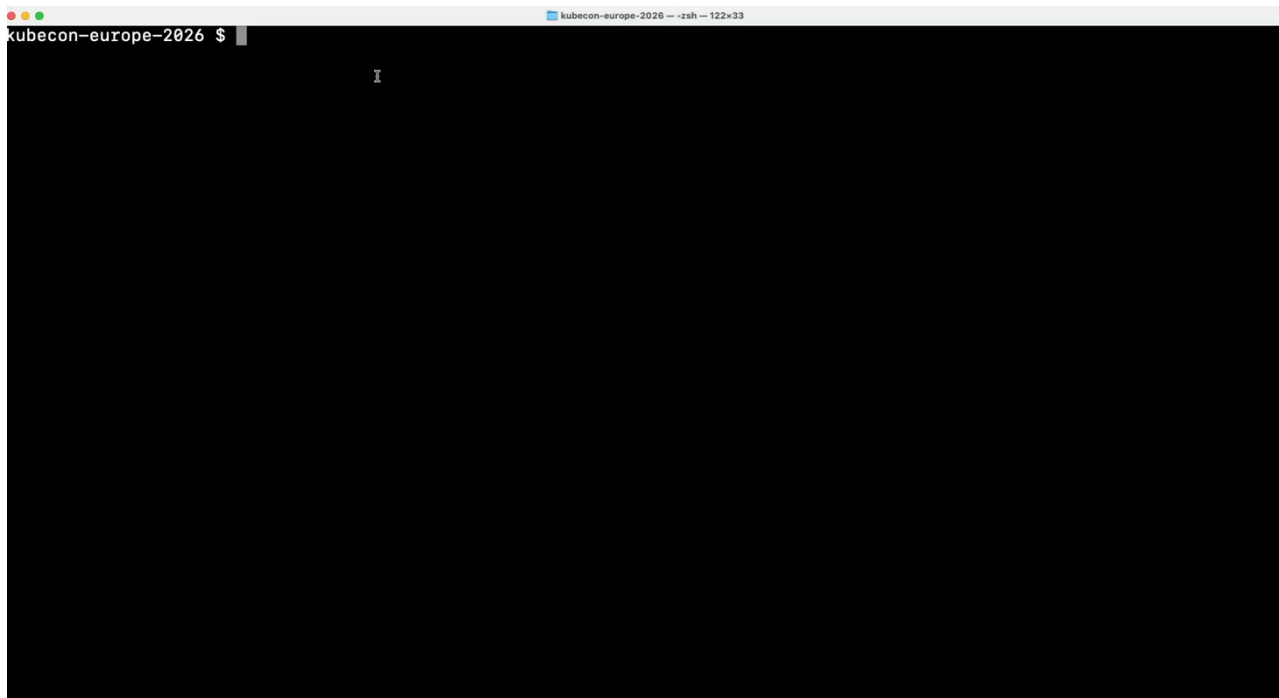
KubeCon



CloudNativeCon

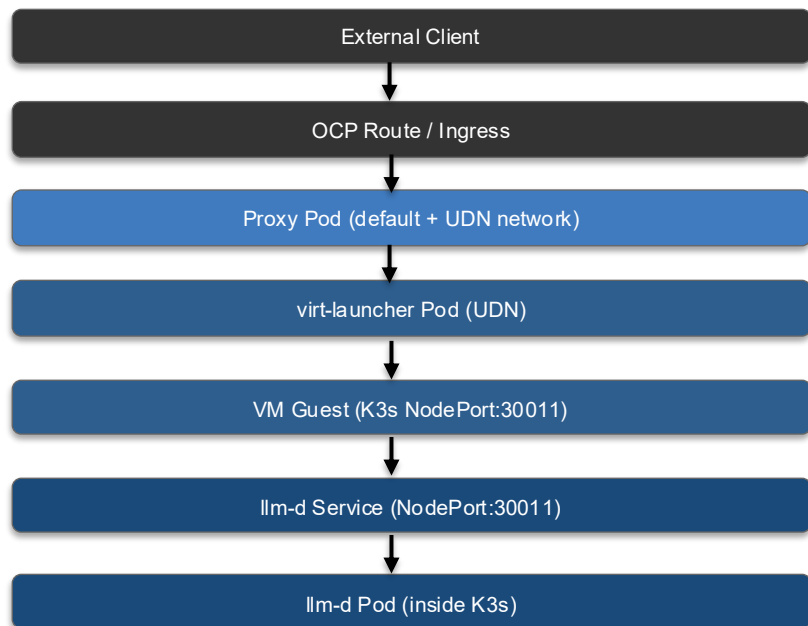
Europe 2026

New Approach: Demo

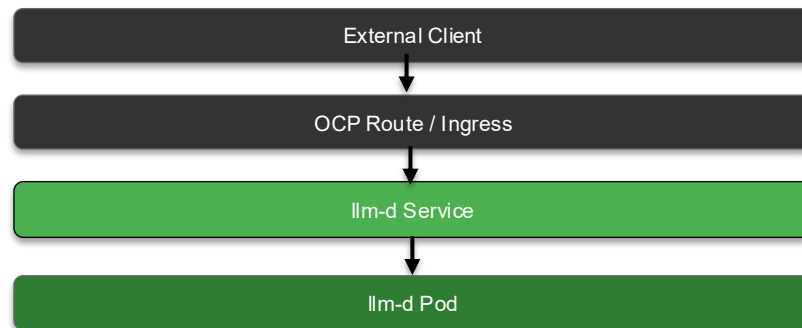


Experiment: Measure Framework Overhead

With UDN/KubeVirt Isolation



Without UDN/KubeVirt Isolation



Experiment: Workload Profile

- **Workload type:** llm-d-sim [1]
- **Load Generator tool:** inference-perf [3]
- Inference-perf Load profile: **Random Dataset + Poisson Load Generator + completion API**

Stages #	Category	Duration (sec)	QPS	Total # Requests
1	Warm-up	10	1	10
2	Low-load	30	5	150
3	Medium-Load	20	10	200

*360 prompts randomly generated with **min**, **avg**, **max** length (tokens): 10, 50, 100*

Experiment: Results – Request Latency



KubeCon



CloudNativeCon

Europe 2026

Stage-1

Req Latency	No isolation (sec)	With Isolation (sec)	Diff (sec)
mean	0.0308	0.0248	-0.0060
min	0.0115	0.0163	0.0048
99 percentile	0.1281	0.0421	-0.0860

Stage-2

Req Latency	No isolation (sec)	With Isolation (sec)	Diff (sec)
mean	0.0140	0.0154	0.0014
min	0.0080	0.0104	0.0024
99 percentile	0.0306	0.0329	0.0023

Stage-3

Req Latency	No isolation (sec)	With Isolation (sec)	Diff (sec)
mean	0.0128	0.0209	0.0081
min	0.0083	0.0100	0.0017
99 percentile	0.0380	0.3550	0.0317



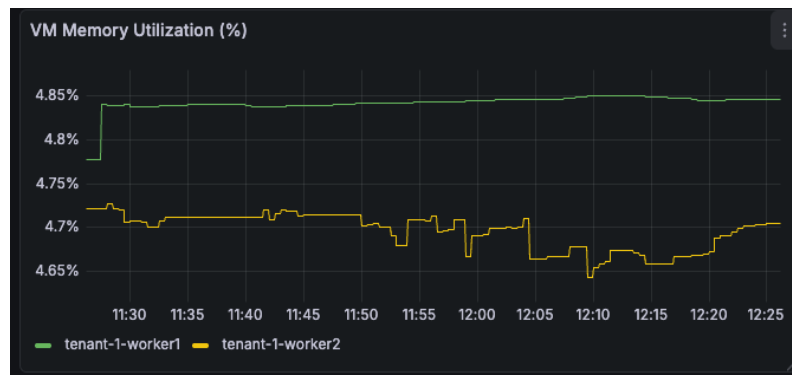
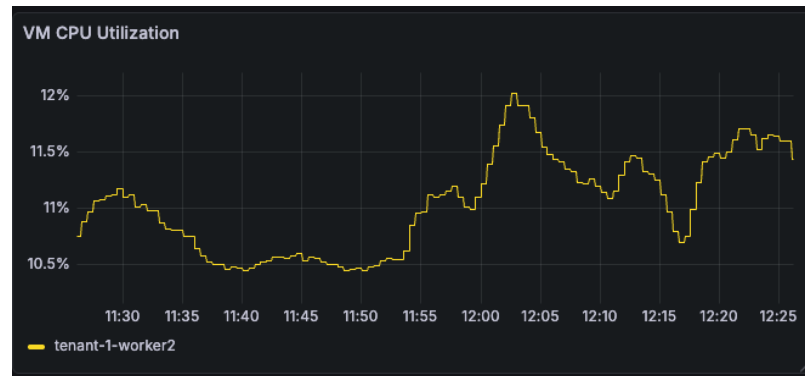
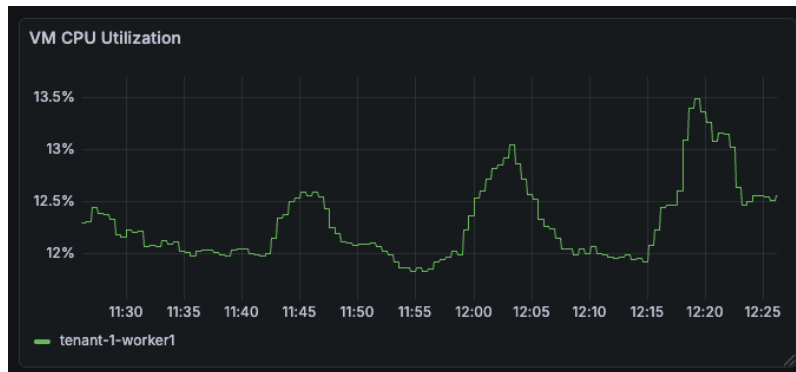
KubeCon



CloudNativeCon

Europe 2026

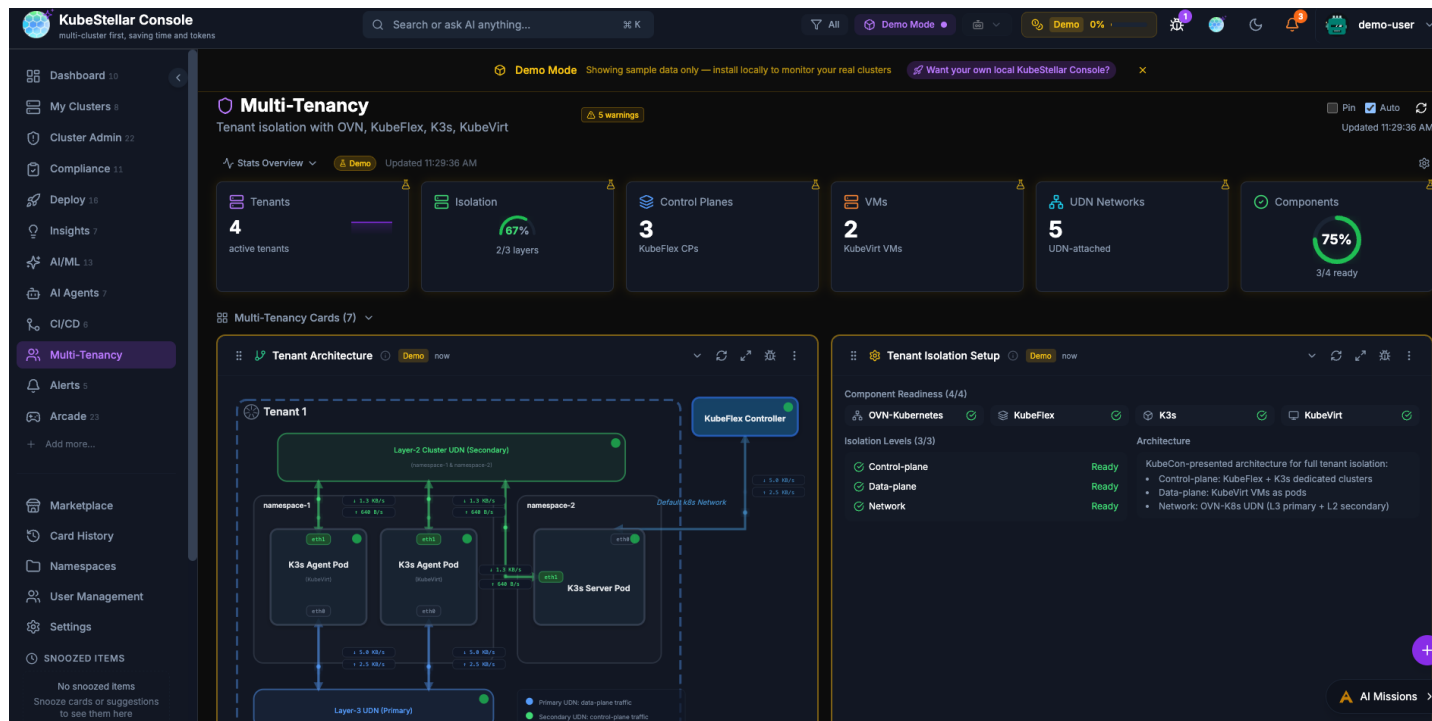
Experiment: Results – Resource Utilization



Kubevirt VM Resource Utilization

On-going Work: Multi-tenancy Console

- Try it: <https://console.kubestellar.io/multi-tenancy>



On-going Work: OpenClaw Isolation



- **OpenClaw: 330K+ Star AI Assistant Platform**
 - Connects to 20+ messaging platforms (WhatsApp, Slack, Signal, Teams...)
 - Runs agent commands on user's host — sandbox is OFF by default
 - Plugins have full host access — no boundary between tools and OS
- **Attack Surface Without Isolation**
 - Credentials at stake: email, health, financial, messages, photos
 - Safety sidecars (e.g. OpenShell,) must not share nodes with general workloads
 - Prompt injection via any messaging channel → host command execution
 - Malicious marketplace plugin → full access to files, env vars, secrets
 - Single bearer token compromise → complete system takeover
 - No multi-tenant authorization — all callers are trusted operators
- **What Three Shades of Isolation Prevents**
 - VM + UDN = isolated trust boundary for agent + its monitor
 - Data-plane: agent runs inside VM — host is unreachable
 - Control-plane: per-tenant API server limits blast radius
 - Network: UDN segments traffic — compromised agent can't reach other tenants

Conclusion & Next Steps

- We presented a new approach for multi-tenancy isolation using only open-source projects
- Our approach hardens tenant's boundaries by providing control-plane, data-plane and network isolation
- Provides simplified management for cluster admins
- **Next Steps**
 - Measure the overhead of our framework across others workload profile
 - Improve our multi-tenancy console that helps to provide a single pane of class for multi-tenancy management for clusters admins



Resources

- [1] <https://github.com/llm-d/llm-d/blob/main/guides/simulated-accelerators/README.md>
- [2] <https://github.com/vllm-project/production-stack>
- [3] <https://github.com/kubernetes-sigs/inference-perf/tree/main>

AI Workloads



kagenti.io



llm-d.ai